

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: MLA03

COURSE NAME: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO5: Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)

Assessment Tool Weightage: 35%

Dr G. A. SENTHIL

QUESTIONS: 10 Total Marks: 50

Annexure A

Simulation-Based Assessment Question

Duration: 1hr

1. Simulate a Hierarchical Reinforcement Learning (HRL) agent for autonomous warehouse navigation.
(5 Marks)
2. Implement a Hierarchical Abstract Machine (HAM) to control a robot performing sequential tasks.
(5 Marks)
3. Develop the MAXQ Value Decomposition algorithm for a taxi navigation environment.
(5 Marks)
4. Simulate a Meta-Reinforcement Learning agent that rapidly adapts to multiple GridWorld environments.
(5 Marks)
5. Design a Multi-Agent Reinforcement Learning (MARL) system for cooperative robot path planning.
(5 Marks)

Code of Conduct:

I K.BALA MURALI(192472010) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

ANSWERS

Answer 1: Hierarchical Reinforcement Learning (HRL) Agent for Autonomous Warehouse Navigation

Approach: The warehouse navigation task is decomposed into a two-level hierarchy - a high-level (meta) controller that selects sub-goals (e.g., "go to shelf A", "go to packing station", "go to charging dock") and a low-level controller that learns primitive actions (move forward, turn left/right, pick, place) to achieve the sub-goal chosen by the meta-controller.

Implementation Steps: 1) Model the warehouse as a grid environment with obstacles, shelves, and drop-off points. 2) Define the option/sub-goal space (each option is a temporally extended action with its own policy, initiation set, and termination condition, following the Options framework). 3) Train the low-level policy using Q-learning/DQN to reach a given sub-goal while avoiding collisions. 4) Train the high-level policy using Q-learning over the option space, where the reward is the cumulative environment reward received while an option executes. 5) Use intrinsic rewards (distance-to-goal shaping) to speed up low-level learning.

Result / Observation: The HRL agent converges faster than a flat RL agent because the high-level policy only has to reason over a small number of sub-goals rather than the full primitive action space, and the low-level skills (navigation, picking) are reusable across multiple high-level tasks. This significantly reduces the effective search space and improves sample efficiency in the warehouse navigation problem.

Answer 2: Hierarchical Abstract Machine (HAM) to Control a Robot Performing Sequential Tasks

Approach: A Hierarchical Abstract Machine (HAM) is used to constrain the robot's policy using a set of finite-state machines instead of learning over the full flat action space. Each abstract machine consists of four types of states: Action states (execute a primitive robot action), Call states (invoke another HAM/sub-machine), Choice states (points where the RL algorithm chooses among alternatives), and Stop states (return control to the calling machine).

Implementation Steps: 1) Decompose the sequential task (e.g., pick -> transport -> place -> return) into separate HAMs, one per sub-task. 2) Encode the known/fixed order of operations directly into the machine structure (Action and Call states), leaving only the genuinely uncertain decisions (e.g., which item to pick next, which path to take) as Choice states. 3) Convert the joint system (environment + HAM) into an induced SMDP (semi-Markov decision process) whose states are the cross-product of environment state and machine state. 4) Apply a standard RL algorithm (e.g., HAMQ-learning / SMDP Q-learning) only at the Choice states to learn the optimal policy.

Result / Observation: Because most of the sequential structure of the task is hand-specified through the machine rather than learned, the robot converges to a reliable task-completion policy with far fewer training episodes than a flat RL agent, and the resulting behavior is easier to verify since the transition order of the sequential task is guaranteed by the HAM structure.

Answer 3: MAXQ Value Decomposition Algorithm for a Taxi Navigation Environment

Approach: The classic Taxi domain task (pick up a passenger from one of four locations and drop them at another) is decomposed into a task hierarchy: Root -> {Get, Put} -> {Navigate(t), Pickup, Dropoff}, where Navigate(t) is further parameterized by the target location t. MAXQ decomposes the value function of the root task into the sum of the value functions of its subtasks (recursive optimality).

Implementation Steps: 1) Define the task graph with subtasks Root, Get, Put, Navigate, Pickup, and Dropoff, each with its own termination predicate and pseudo-reward function. 2) For each subtask, decompose its value function as $V(\text{subtask}) = C(\text{parent}, \text{subtask}) + V(\text{child})$, where C is the completion function representing the expected reward of completing the parent task after finishing the child subtask. 3) Learn the completion function values using a temporal-difference update rule at every level of the hierarchy simultaneously as the taxi interacts

with the environment. 4) Use the recursively optimal policy derived bottom-up: choose the best Navigate target, then decide whether to Pickup/Dropoff, then choose Get vs Put at the root.

Result / Observation: The MAXQ decomposition allows subtasks like Navigate(t) to be learned once and reused across both the Get and Put branches (state abstraction), which drastically reduces the effective state space compared to flat Q-learning and produces faster convergence to an optimal (or recursively optimal) pickup-and-dropoff policy in the Taxi environment.

Answer 4: Meta-Reinforcement Learning Agent that Rapidly Adapts to Multiple GridWorld Environments

Approach: A meta-RL agent is trained across a distribution of GridWorld tasks (varying goal positions, wall layouts, or reward locations) so that it learns a fast adaptation strategy ('learning to learn') rather than a policy specialized to a single grid. A recurrent policy (RNN/LSTM-based actor-critic) or a gradient-based meta-learning method such as MAML is used.

Implementation Steps: 1) Sample a distribution of GridWorld tasks that share the same state/action structure but differ in goal location, obstacle placement, or reward function. 2) For the recurrent meta-RL approach, feed the agent's recent observations, actions, and rewards as recurrent state, allowing it to implicitly infer the current task from a handful of interaction steps. 3) Train the agent with an actor-critic algorithm (e.g., A2C/PPO) across many sampled GridWorld tasks so the recurrent hidden state learns to encode task-identifying information. 4) Alternatively, use MAML: perform a few gradient steps of adaptation per sampled task (inner loop), then update the shared initialization parameters based on post-adaptation performance across tasks (outer loop). 5) Evaluate on held-out GridWorld configurations to measure adaptation speed.

Result / Observation: The meta-trained agent needs only a handful of episodes (often fewer than 5-10) in a new, previously unseen GridWorld layout to identify the goal location and near-optimal path, compared to a standard RL agent trained from scratch, which needs hundreds of episodes - demonstrating the sample-efficiency benefit of meta-RL for rapidly changing environments.

Answer 5: Multi-Agent Reinforcement Learning (MARL) System for Cooperative Robot Path Planning

Approach: Multiple robots must jointly plan collision-free paths to their individual goals while cooperating (e.g., yielding right-of-way, avoiding congestion). This is modeled as a cooperative Markov game with a shared/team reward that penalizes collisions and rewards reaching goals efficiently. A Centralized Training with Decentralized Execution (CTDE) approach such as MADDPG or QMIX is used.

Implementation Steps: 1) Model each robot as an agent with a local observation (its position, nearby obstacles, and relative positions of nearby robots) and a continuous or discrete action space (movement direction/velocity). 2) Define a shared cooperative reward: positive reward for reaching individual goals, negative reward for collisions or path overlaps, and a small step penalty to encourage efficiency. 3) During training, use a centralized critic that has access to the joint state and joint actions of all robots (CTDE), while each robot's actor only uses its own local observation - enabling coordinated learning without requiring full communication at execution time. 4) Train using an algorithm such as MADDPG (continuous control) or QMIX/VDN (value decomposition for discrete grid movement) so the team-level value function can be factorized into per-agent contributions. 5) Test the trained decentralized policies in denser robot configurations to evaluate scalability.

Result / Observation: The cooperative MARL system learns implicit coordination behaviors, such as robots yielding at narrow corridors or taking alternate routes, resulting in a significantly lower collision rate and shorter average time-to-goal compared to independent (non-cooperative) path-planning agents that ignore each other's presence.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	25	Demonstrates deep understanding of concepts with complete clarity	Explains concepts correctly with minor gaps	Partial understanding with errors or omissions	Unable to explain basic concepts
Application	25	Effectively applies concepts to new situations; solves problems logically	Applies concepts with minor errors	Limited ability to apply concepts; requires guidance	Unable to apply concepts effectively
Analytical Thinking	20	Critically analyzes scenarios with strong justification and reasoning	Provides reasonable analysis with some justification	Superficial analysis with weak reasoning	No analytical reasoning demonstrated
Communication	15	Communicates ideas clearly, confidently, and with appropriate terminology	Communicates clearly but with minor hesitation	Communication lacks clarity or structure	Unable to communicate ideas effectively
Response to Questions	15	Responds accurately and confidently, extending answers with depth	Responds correctly but with limited depth	Responds partially; requires prompting	Unable to respond to questions effectively